

ECS 408/608 : Operating System

Introduction to Linux Tools

Instructor: Dr. Sukarn Agarwal,
Assistant Professor,
EECS Department,
IISER Bhopal

January 13, 2026

Abstract

In this assignment, you have to run a few simple Linux commands on the shell to understand the basics of OS. In each problem statement, you are expected to run a basic command and report the result you observe on the screen. Furthermore, report the tools that you used while solving the exercise. Different helper files: `memory_1.c`, `memory_2.c`, `disk_1.c`, `disk_2.c`, `make-copies.sh`, `foo.pdf`, `cpu.c` and `cpu_print.c` are given to you for solving this assignment.

Prerequisite

- You must be comfortable writing / compiling / creating / copying / moving / viewing the file using the Linux command from the terminal. In case you are not familiar with the basic Linux commands, learn the basic syntaxes using the following link:

<https://www.tutorialspoint.com/unix/what-is-linux.htm>

- You need to know the basic syntax of compilation and running the C program from the graphical IDE and the command line. With the editor of your choice, type the C program and compile and run the `.c` file using the following commands:

```
$ gcc hello_world.c
$ ./a.out
$ gcc hello_world.c -o hello
$ ./hello
```

- You need to know how to tar and untar files.

```
tar -xvzf intro-code.tgz
```

- Understand the common tools of Linux like `top` and `ps` commands. These tools are basically used to monitor the process running in the system.

- Understand the `/proc` filesystem of Linux. The `/proc` file system is a mechanism the Linux operating system provides to report information about the system and process to users. To get more information about the `proc`, run the command `man proc`. Furthermore, learn the system wide `proc` files such as `meminfo`, `status`, `stat`, `limits`, `cpuinfo` and `maps`.

Problems

1. With this problem, we will understand the status of the running process using the `top` command. To do this, compile the program `cpu.c` and run it on bash or any other terminal of your choice as follows:

```
gcc cpu.c -o cpu
./cpu
```

This program executes an infinite loop. Now open another window of the terminal screen and, run the `top` command, and answer the following question about the `cpu` process.

- (a) Determine the PID of the process executing the CPU program.
 - (b) How much memory and CPU does this process consume?
 - (c) Determine the current status of the process. For instance, is it executing in a blocked, active, or zombie state?
2. In this problem, you are going to understand the hardware configuration of your Linux machine using the `/proc` filesystem. Execute command `more /proc/cpuinfo` and explain the following terms: `cores` and `processors`. Furthermore, use the command `lscpu` to verify your definitions. Before attempting this question, you need to understand the concept of CPU hyperthreading at a high level.
 - (a) How many `cores` does your machine have?
 - (b) How many `processors` does your machine have?
 - (c) Determine the frequency of each processor.
 - (d) Determine how much physical memory your system has.
 - (e) Determine the architecture of CPU?
 - (f) How much of the memory is occupied?
 - (g) Determine the total number of forks and context switches since your system booted up.
 3. Compile and execute the two programs `memory_1.c` and `memory_2.c` given to you. These programs allocate a large array in memory. The only difference is one program accesses the array, and the other doesn't. Note that both programs pause before exiting to let you inspect their memory usage. With the `ps` command, you can inspect the memory used by these programs. For instance, the output will tell you the total size of the

“virtual” memory of the process is, and how much of this is actually physically resident in memory. With that, you will learn the virtual memory allocated to the process that the process thinks it has. Whereas the OS only allocates a subset of this memory physically in RAM.

Compare both programs’ virtual and physical memory usage, and explain your observations.

4. In this problem, you will understand how the Linux shell (e.g., bash shell) executes user commands by spawning new child processes to execute the various commands.

- (a) Compile the program `cpu-print.c` given to you and execute it in the `bash` or any other shell of your choice as follows.

```
gcc cpu-print.c -o cpu-print
./cpu-print
```

This program executes in an infinite loop, printing output to the screen. Now, open another terminal and use the `ps` command with suitable options to find out the `pid` of the process spawned by the shell to run the `cpu-print` executable. You may want to explore the `ps` command thoroughly to understand its various output fields.

- (b) Find the PID of the parent of the `cpu-print` process, i.e., the shell process. Next, find the PIDs of all the ancestors, going back at least five generations (or until you reach the limit process).
 - (c) You will now understand how the shell performs output redirection. Execute the following command.

```
./cpu-print > /tmp/tmp.txt &
```

Look at the `proc` file system information of the newly spawned process. Please pay particular attention to where its file descriptors 0, 1, and 2 (standard input, output, and error) are pointing to. Using this information, can you describe how Shell is implementing I/O redirection?

- (d) With this subproblem, you will learn how to implement pipes using the shell. For this, execute the following command:

```
./cpu-print | grep hello &
```

One more time, find the newly spawned processes and look out for the standard input/output. Error file descriptors are pointing to; with this, explain how pipes are implemented in the shell.

- (e) When you type a command in the shell, the shell performs two things. For some commands, binary files that perform that functionality already come with your Linux kernel installation. For such an operation, the shell invokes the binary like it executes the binaries of your own programs. The shell implements the command within its code for other commands where the executable or binaries do not exist.

Consider the following command that you can type in bash shell: `cd`, `ls`, `history`, `ps`. Determine which of these commands already exist as executables in the Linux kernel directory tree, which are then simply executed by the bash shell and implemented by the bash code itself.

5. In this problem, you will compile and execute the program `disk_1.c` and `disk_2.c`. These programs read a large number of files from the disk. To do so, you first create these files as follows: Create a folder `disk-files` and place `foo.pdf` file in it. Consequently, use the script `make-copies.sh` (place outside the `disk-files` folder) to make 5000 copies of the same file in folder `disk-files`. The disk programs will read these files. Now, execute the disk programs one after another. With each program, measure the disk utilization while the program is executing on the system. To measure the disk utilization, use the `iostat` tool (You can find the usage of `iostat` with `man iostat` command). Report and explain all your observations. Along with that, read the code of the program in order to explain your observation.

Hint: Note that for this exercise to work correctly, you must be reading from a directory on the local disk. If your `disk-files` directory is not on a local disk (but, say, mounted via NFS), then you must alter the location of the files in the code provided to you to enable reading from a local disk. Also, modern operating systems store recently read files in a cache in memory (called disk buffer cache) for faster access to the same files in the future. In order to ensure that you are making observations while actually reading from disk, you must clear your disk buffer cache between multiple runs of `disk_1.c`. If you do not clear the disk buffer cache between successive runs of `disk_1.c`, you will be reading the files not from disk but from memory. Look up online for commands on how to clear your disk buffer cache, and note that you will need superuser permissions to execute these commands

6. Different user programs make several system calls (which are most likely the function call to OS, but more complicated) to invoke OS functionality during their execution. The `strace` tool let you trace the system calls invoked by a running executable. Consider any executable in the example above or any Linux command like `ls` and execute with the `strace` command as follows:

```
strace <executable>
```

This command shows the list of all system calls made by the executable during its run. How many system calls does the program make in total? Look up online to find the list of system calls supported by your operating system, and see if you can locate any of them in the output of `strace` command.

Assignment Submission Instruction:

- Based on your observed result, create a report with your name and roll number with assignment ID (For example, `Sukarn_Agarwal_21056_Assign1.pdf`).

- Submitted pdf file contains answers to all the questions above in order. **Any out-of-order answer results in a zero mark.**
- With each question, attach all the screenshots you observed on your screen. Make sure that your name should appear there. **If TA finds any discrepancy in this, zero marks will be awarded.**
- **The last date to submit the assignment is by the end of day of 27 Jan 2026 (IST). Any late submission results in the deduction of 50% Marks.**